

CI/CD

Para esta sección, necesitaremos que crees los pipelines para la implementación de la aplicación descrita anteriormente. Puedes elegir la herramienta de tu preferencia, Jenkins File, MakeFile, Code Pipeline, GitHub Actions u otra de tu preferencia, eres libre de elegir la herramienta que te resulte más cómoda. No obstante, debes proporcionar algunos aportes como beneficios o desventajas de la herramienta que elijas. Para el pipeline en sí, queremos que:

1. Crees un pipeline que permita la implementación de sus aplicaciones en tres ambientes distintos (dev, stg, prod) sobre tres regiones diferentes (virginia, Ohio y oregon), es decir la aplicación sobre un ambiente específico debe ser desplegada sobre una región determinada (ejemplo dev sobre Ohio). Esto debe ser parametrizable (ambientes y regiones) desde el archivo de propiedades.
2. Describa y configure 2 estrategias de despliegues y actualización de aplicaciones sobre contenedores en AWS (fargate), detalle las ventajas de cada una de estas estrategias, y en qué tipo de aplicaciones es beneficioso o no utilizar una u otra. Recuerde documentar y evidencias las pruebas realizadas y deje como variables parametrizables el poder utilizar una estrategia u otra en su código IaC.

Solución propuesta:

CICD usando Github Actions. Esquema general:

1. Crear cuentas independientes para DEV, STG y PROD en github (Finaktiva_dev, Finaktiva_stg y Finaktiva-prd). Motivo: Seguridad, minimizar errores e independencia entre grupos de trabajo.
2. Crear 3 ambientes similares a servidor de Terraform antes descrito para el despliegue cicd de infraestructura IaaS para toda la infraestructura del cliente (AWS, Azure, Google, On-Premise, etc). Independientes 100% (cuentas, usuarios, configuración, etc). Los servidores serán:
 - a. CICD_DEV (para el grupo desarrollador)
 - b. CICD_STG (para el grupo de pruebas)
 - c. CICD_PROD (para Operaciones).
3. En AWS existirán 3 tenances independientes. Por seguridad y para evitar errores: uno por cada ambiente, aislados y no interconectados.
 - a. Tenance_DEV (virginia)
 - b. Tenance_STG (Ohio)
 - c. Tenance_PROD (Oregon)
4. Posible uso de AWS Tower para manejo de cuentas Maestras y subcuentas (dev,stg,prod). Recursos, Facturación, Performance, separados por: DEV, STG, PROD.
5. Cuentas AWS para IaC. Detalle general:
 - a. Inicialmente para provisioning de infraestructura se requerirá el role FULL_ADMIN.
 - b. La cuenta estará vinculada a una región única y a un grupo único (dev, stg, prod). Y será asignada a los grupos de trabajo respectivos.
 - c. Para Prod se propone, para la segregación funcional de cicd (infra, app, redes, etc.) el uso de carpetas independientes en Terraform (ejemplo: /terraform/aws/vpc (para redes), /terraform/aws/eks (para Kubernetes), /terraform/aws/app (para aplicaciones), /terraform/azure (para Cloud Microsoft), /terraform/ibm (para IbmCloud), /terraform/onpremise para servidor físicos)
6. Control y pases entre ambientes DEV/STG/PROD. Detalle general:

- a. Los distintos ambientes usaran repositorios github como fuentes de configuración y binarios de aplicativos
- b. El pase de DEV > STG > PROD sera via git clone de repositorios (copias).
- c. Los pipelines estarán desplegados en github action (IaaS, app, etc).

Fargate. Esquema general:

7. AWS recomienda Fargate para procesos no persistentes en tiempo con tiempo de ejecución de pocos minutos (15 minutos o menos).
 - a. Aplicaciones tipo: web, API y microservicios (no persistentes)
 - b. Procesamiento de datos (tareas que se ejecuten: <15min, 16 vCPUs y 120 GB).
 - c. Serverless. Sin administración SO
 - d. Escalables automáticamente.
 - e. Costos. Basicamente por uso trafico de usuarios y procesamiento
 - f. No apto para aplicativos que requieren 100% persistencia y atención online. Ejemplos: Bases de Datos, Aplicaciones persistentes (ej: AD, DNS, etc).
8. Despliegues. Anexo se plantean 2 ejemplos de despliegues con las siguientes características:
 - a. En AWS usando Terraform
 - b. Las plantillas iniciales (ejemplos) fueron creadas en **Virtual Studio Code** (Microsoft) utilizando el plugin de Amazon-Q (IA con orientación a documentación ejemplos de AWS).
 - c. Ejemplo 01: Simple Web App (yaml anexo)
 - d. Ejemplo 02: Microservicios App (yaml anexo).
9. Estrategias:
 - a. Herramienta_01. Uso de git/github/github-actions
 - i. Repositorio compartido
 - ii. Repositorios en Terraform divididos por función-filesystem (ordenes de cambio, aplicativos, etc)
 - b. Herramienta_02. Uso de IDE (Entorno de Desarrollo Integrado)
 - i. Se plantea **Visual Studio Code** de Microsoft (VS-code) para todos los grupos
 - ii. **Extensiones VS-code Generales** (para todos los usuarios): Amazon-Q, Github Copilot, Draw.io (diagramas), git (vinculado a una cuenta github), etc.
 - iii. **Extensiones VS-code para IaaS**
 - AWS
 - YAML
 - Dockers
 - iv. **Extensiones VS-code para programadores**
 - C/C++/C# lenguaje
 - Python
 - Dot Net, etc
 - c. **Despliegues por control de versiones en git**
 - i. Uso 100% de versionamiento por todos los miembros del grupo
 - ii. Ventajas: Disponibilidad 100% de versiones estables
 - iii. Desventajas: Los errores en código se propagan con mayor facilidad
 - d. **Despliegues por control de versiones en carpetas de terraform**
 - i. Uso 100% de carpetas de proyectos en terraform
 - ii. Ventajas: se acota/aisla una sección de desarrollo a una sola carpeta
 - iii. Desventajas: Administración de múltiples carpetas organizadas por: app / redes / IaaS / web / databases / etc

